

Additional monad combinators

Listing 13.3 makes use of some monad combinators we haven't seen before, although they can be defined for any `Monad`. You may want to think about what these combinators mean for types other than `IO`. Note that not all of them make sense for every monadic type. (For instance, what does `forever` mean for `Option`? For `Stream`?)

```
def doWhile[A](a: F[A])(cond: A => F[Boolean]): F[Unit] = for {
  a1 <- a
  ok <- cond(a1)
  _ <- if (ok) doWhile(a)(cond) else unit(())
} yield ()
```

Repeats the effect of the first argument as long as the `cond` function yields `true`

```
def forever[A,B](a: F[A]): F[B] = {
  lazy val t: F[B] = forever(a)
  a flatMap (_ => t)
}
```

Repeats the effect of its argument infinitely

Folds the stream with the function `f`, combining the effects and returning the result

```
def foldM[A,B](l: Stream[A])(z: B)(f: (B,A) => F[B]): F[B] =
  l match {
    case h #:: t => f(z,h) flatMap (z2 => foldM(t)(z2)(f))
    case _ => unit(z)
  }
```

The same as the `foldM` function above except ignores the result

```
def foldM_[A,B](l: Stream[A])(z: B)(f: (B,A) => F[B]): F[Unit] =
  skip { foldM(l)(z)(f) }
```

```
def foreachM[A](l: Stream[A])(f: A => F[Unit]): F[Unit] =
  foldM_(l)()( (u,a) => skip(f(a)) )
```

Calls the function `f` for each element of the stream and combines the effects

We don't necessarily endorse writing code this way in Scala.⁴ But it does demonstrate that FP is not in any way limited in its expressiveness—every program can be expressed in a purely functional way, even if that functional program is a straightforward embedding of an imperative program into the `IO` monad.

13.2.2 Benefits and drawbacks of the simple IO type

An `IO` monad like what we have so far is a kind of least common denominator for expressing programs with external effects. Its usage is important mainly because it

⁴ If you have a monolithic block of impure code like this, you can always just write a definition that performs actual side effects and then wrap it in `IO`—this will be more efficient, and the syntax is nicer than what's provided using a combination of for-comprehension syntax and the various `Monad` combinators.

clearly separates pure code from impure code, forcing us to be honest about where interactions with the outside world are occurring. It also encourages the beneficial factoring of effects that we discussed earlier. But when programming *within* the `IO` monad, we have many of the same difficulties as we would in ordinary imperative programming, which has motivated functional programmers to look for more composable ways of describing effectful programs.⁵ Nonetheless, our `IO` monad does provide some real benefits:

- `IO` computations are ordinary *values*. We can store them in lists, pass them to functions, create them dynamically, and so on. Any common pattern can be wrapped up in a function and reused.
- Reifying `IO` computations as values means we can craft a more interesting interpreter than the simple `run` method baked into the `IO` type itself. Later in this chapter, we'll build a more refined `IO` type and sketch out an interpreter that uses non-blocking I/O in its implementation. What's more, as we vary the interpreter, client code like the `converter` example remains identical—we don't expose the representation of `IO` to the programmer at all! It's entirely an implementation detail of our `IO` interpreter.

Our naive `IO` monad also has a few problems:

- Many `IO` programs will overflow the runtime call stack and throw a `StackOverflowError`. If you haven't encountered this problem yet in your own experimenting, you'd certainly run into it if you were to write larger programs using our current `IO` type. For example, if you keep typing numbers into the `factorialREPL` program from earlier, it eventually overflows the stack.
- A value of type `IO[A]` is completely opaque. It's really just a lazy identity—a function that takes no arguments. When we call `run`, we hope that it will eventually produce a value of type `A`, but there's no way for us to inspect such a program and see what it might do. It might hang forever and do nothing, or it might eventually do something productive. There's no way to tell. We could say that it's *too general*, and as a result there's little reasoning that we can do with `IO` values. We can compose them with the monadic combinators, or we can run them, but that's all we can do.
- Our simple `IO` type has nothing at all to say about concurrency or asynchronous operations. The primitives we have so far only allow us to sequence opaque blocking `IO` actions one after another. Many I/O libraries, such as the `java.nio` package that comes with the standard libraries, allow non-blocking and asynchronous I/O. Our `IO` type is incapable of making use of such operations. We'll rectify that by the end of this chapter when we develop a more practical `IO` monad.

Let's start by solving the first problem (overflowing the call stack), since it will inform our solution for the other two.

⁵ We'll see an example of this in chapter 15 when we develop a data type for composable streaming I/O.